

URD Guardian (Jarvis)

Người gác cổng giữa Yêu cầu và Mã nguồn — bằng AI

Một AI agent đọc git, mã nguồn và tài liệu URD (User Requirements Document), rồi dùng LLM để đánh giá mỗi commit có thực sự thỏa mãn các tiêu chí chấp nhận hay không.

Node.js • Không thư viện runtime bên ngoài • Đa dự án • Tất cả tài liệu là Markdown

Vấn đề

Trong hầu hết các nhóm phát triển phần mềm:

- 📄 Tài liệu và mã nguồn trôi dạt khỏi nhau. URD viết một đàng, code làm một nẻo. Không ai biết cho tới khi quá muộn.
- 🔍 Không ai kiểm chứng commit có đáp ứng tiêu chí chấp nhận không. Reviewer đọc code, nhưng hiếm khi đối chiếu ngược lại từng acceptance criteria trong URD.
- 📝 Tài liệu sử dụng luôn lỗi thời. Viết tay tốn công, và không ai cập nhật sau khi code đổi.
- 💎 Thiếu bức tranh phủ (coverage). Không rõ yêu cầu nào đã xong, đang làm, hay chưa động tới.

Hệ quả: yêu cầu bị bỏ sót, thay đổi lộn lạo, tri thức dự án phân mảnh.

Giải pháp

URD Guardian kết nối ba nguồn sự thật — Git ↔ Mã nguồn ↔ URD — và dùng LLM làm "reviewer".

Với mỗi commit tham chiếu `REQ-xxx`, agent:

- Đọc diff và nội dung đầy đủ các file bị thay đổi.
- Nạp yêu cầu tương ứng cùng các **Acceptance Criteria**.
- Gửi cho LLM để chấm **PASS / FAIL** trên từng tiêu chí, kèm lý do.
- Ghi kết quả vào registry để dashboard và CLI dùng chung.

Ngoài ra agent còn tự sinh tài liệu, chấm điểm chất lượng URD, và trực quan hóa mọi thứ.

Kiểm tra commit bằng LLM

Đây là trái tim của hệ thống (`commit-checker.js` + `llm-checker.js`):

- Trích mọi `REQ-xxx` / `CR-xxx` từ commit message.
- Với mỗi yêu cầu được claim, gọi LLM chấm từng acceptance criteria.
- Nguyên tắc chấm: **PASS** nếu code có vẻ hiện thực tiêu chí; **FAIL** chỉ khi có bằng chứng rõ ràng là không đạt; thiếu bằng chứng thì ưu tiên PASS.

Các loại ghi chú (notes):

- `requirement-satisfied` (PASS) · `requirement-not-satisfied` (FAIL)
- `unknown-requirement` — claim `REQ-ID` không có trong URD
- `no-claim` — commit đổi code nhưng không tham chiếu yêu cầu nào
- `llm-unchecked` — LLM chưa cấu hình hoặc không khả dụng

Commit `ok = false` nếu tồn tại bất kỳ note cấp `error` .

Sinh tài liệu tự động

Lệnh `generate-docs` quét `src/` , trích symbol (hàm, class, endpoint) và ghi **5 tài liệu**

Markdown vào `docs/usage/` :

File	Đối tượng
<code>overview.md</code>	Mọi người — ứng dụng là gì, tính năng, khởi động nhanh
<code>user-guide.md</code>	Người dùng cuối — cách dùng từng tính năng kèm ví dụ curl
<code>api-reference.md</code>	Lập trình viên — endpoint, module, hàm
<code>developer-guide.md</code>	Lập trình viên — kiến trúc, cách mở rộng
<code>index.md</code>	Mục lục điều hướng

Tài liệu được sinh từ URD + mã nguồn, không cần viết tay.

URD thông minh với LLM

Ngoài kiểm commit, LLM còn hỗ trợ chính tài liệu yêu cầu:

- 🗉 **Review URD** (`/api/urd/review`) — chấm điểm chất lượng URD: định dạng, độ rõ ràng, acceptance criteria, tính nhất quán ID, và các thiếu sót; trả về điểm số + điểm mạnh/yếu + gợi ý.
- ✨ **Chuẩn hóa URD** (`/api/urd/standardize`) — tự sinh 2–5 acceptance criteria cụ thể, kiểm chứng được cho mỗi yêu cầu còn sơ sài.

- 📦 Tải URD lên trực tiếp qua dashboard, tự động sync vào registry.

LLM chạy qua VNG Cloud MaaS API, cấu hình bằng biến môi trường `LLM_API_KEY` , `LLM_BASE_URL` , `LLM_MODEL` .

Truy vết & Đo lường

Dashboard cung cấp các góc nhìn quản trị:

- 🔗 **Traceability matrix** (`/api/traceability`) — yêu cầu ↔ commit ↔ file nguồn ↔ file test, phát hiện test liên quan theo REQ-ID .
- 📈 **Coverage** (`/api/coverage`) — tỷ lệ yêu cầu PASS / FAIL / chưa bắt đầu, thống kê theo từng URD, và đường xu hướng phủ theo thời gian.
- 🎯 **Impact analysis** (`/api/impact`) — mỗi file URD ảnh hưởng tới yêu cầu nào, trạng thái hiện thực ra sao.

Quản lý nhiều dự án

URD Guardian là một server đa dự án:

- ➕ **Thêm dự án** bằng đường dẫn local, hoặc **clone trực tiếp từ git URL** (kèm repo URD riêng nếu có).
- 📁 Mỗi dự án có **registry riêng** (yêu cầu ↔ commit ↔ notes).
- 🔍 **Quét toàn bộ lịch sử commit** trong nền (`/api/check/all`) với theo dõi tiến độ real-time.
- 🪝 **Git hook** — tải script `commit-msg` để **chặn commit** không khớp URD ngay trên máy dev (`git commit --no-verify` để bỏ qua).

Trải nghiệm sử dụng

Chỉ với vài lệnh CLI:

```
bash
node src/index.js init           # tạo docs/urd, docs/usage, data
node src/index.js sync           # nạp yêu cầu URD vào registry
node src/index.js check recent   # kiểm 10 commit gần nhất bằng LLM
node src/index.js check <sha>    # kiểm một commit cụ thể
node src/index.js generate-docs  # sinh 5 tài liệu vào docs/usage/
node src/index.js dashboard      # mở http://localhost:7171
```

Quy ước commit để agent biết commit thực thi yêu cầu nào:

```
plaintext
feat: add login endpoint    Req: REQ-001, REQ-002
[REQ-001] add login endpoint
```

Kiến trúc & Công nghệ

- Node.js >= 18 — nền tảng chạy.
- Không thư viện runtime bên ngoài — chỉ module built-in (http , https , fs , path , child_process , crypto , url).
- Git CLI — đọc lịch sử, diff, clone dự án.
- LLM qua VNG Cloud MaaS — chấm PASS/FAIL, review & chuẩn hóa URD.

```
plaintext
src/
├─ index.js          # CLI entry
├─ server.js / deploy.js
├─ agent/            # git-reader, urd-reader, source-reader, commit-checker,
│                     # llm-checker, doc-generator, version-manager
├─ store/            # fs-utils, registry db, projects (đa dự án)
├─ dashboard/        # http server + public/index.html
docs/ → urd/ (nguồn sự thật) + usage/ (tự sinh)
data/ → registry theo dự án + scan-progress/
```

Điểm khác biệt

Tiêu chí	Cách truyền thống	URD Guardian
Đối chiếu code ↔ tiêu chí	Thủ công, dễ bỏ sót	LLM chấm từng AC mỗi commit
Tài liệu sử dụng	Viết tay, lỗi thời	Sinh 5 tài liệu từ URD + code
Chất lượng URD	Không ai kiểm	LLM review + tự sinh AC
Truy vết	Rời rạc	Traceability + coverage + impact
Chặn lỗi	Sau khi merge	Git hook chặn ngay lúc commit
Quy mô	Một repo	Đa dự án, clone từ git

Đối tượng & Giá trị

Dành cho ai:

- Nhóm phát triển đề cao truy vết yêu cầu (regulated, enterprise, sản phẩm phức tạp).
- Team muốn đưa AI vào khâu kiểm soát chất lượng theo yêu cầu.
- Reviewer, tech lead, QA cần bằng chứng "code khớp acceptance criteria".

Giá trị mang lại:

- ☒ LLM chấm từng tiêu chí — bắt sớm yêu cầu chưa đạt.
 - ☒ Tài liệu và acceptance criteria luôn tươi mới, tự sinh.
 - ☒ Nhìn thấy độ phủ, truy vết, và tác động thay đổi theo thời gian.
 - ☒ Chặn commit lệch chuẩn ngay trên máy dev.
-

Thành viên team Jarvis

- Phùng Văn Minh
 - Lê Văn Tùng
 - Đỗ Quốc Dũng
 - Triệu Văn Dũng
 - Đặng Hữu Hoàn
-

URD Guardian

Giữ mã nguồn trung thực với yêu cầu — bằng AI, tự động, minh bạch.

Bắt đầu ngay:

```
bash
node src/index.js init && node src/index.js sync
node src/index.js dashboard          # http://localhost:7171
```

Node.js >= 18 • MIT License • Cần LLM_API_KEY để bật chấm điểm bằng AI